LBRO Engineering Handbook

Interview · Placement · Demonstration · Viva · Presentation

Based on the LBRO v1.0 Repository — Source-Verified

August 2026

Contents

1 TABLE OF CONTENTS

2 SECTION 1 — PRODUCT OVERVIEW

2.1 What is LBRO?

LBRO stands for **Law-aware Breach Response Orchestrator**. It is a full-stack, production-grade security incident response platform. When a security event happens — a DDoS attack, SQL injection attempt, port scan, malware infection — LBRO captures it, classifies it using machine learning, stores forensic evidence, calculates your regulatory compliance obligations, and helps your team investigate and close the incident.

Think of it as the **control centre for your security team** after something goes wrong.

2.2 What Problem Does It Solve?

Most companies, when they get hacked or detect an attack, face three simultaneous problems:

1. **Operational chaos** — Who knows about this? Who is investigating? What has been done so far?
2. **Evidence loss** — Log files get overwritten, screenshots get lost, nobody captures the network trace.
3. **Regulatory panic** — If customer data was exposed, GDPR says you have 72 hours to notify authorities. HIPAA gives 60 days. What are YOUR obligations right now?

LBRO solves all three in one platform.

2.3 Why Was It Built?

Security teams, especially in startups and mid-market companies, use a fragmented set of tools: spreadsheets for tracking, email for communication, personal cloud drives for evidence, and Google searches to figure out their compliance deadlines. LBRO replaces all of that with a single integrated platform that:

- Ingests incidents from any source via REST API or SDK
- Classifies attacks automatically using ML
- Locks evidence in a tamper-evident vault with chain-of-custody logging
- Calculates compliance deadlines (GDPR 72h, HIPAA notification, DPDPA 72h) automatically
- Generates weekly security reports and compliance audit PDFs with one click

2.4 Who Uses LBRO?

Role	What They Do in LBRO
Security Analyst	Investigates incidents, uploads evidence, writes notes, closes incidents
Admin	Manages team members, projects, views all data across the platform
Viewer	Read-only access — useful for auditors, managers, clients
Super Admin	Platform operator — manages the entire LBRO deployment
Developers	Integrate their applications via the SDK to auto-report events

2.5 Product Vision

LBRO's long-term vision is to be **the security operations layer** that sits between a company's applications and their compliance/legal obligations. Instead of hiring a dedicated compliance officer or a security operations centre, a small engineering team can integrate LBRO and get:

- Automated incident triage
- Real-time regulatory deadline tracking
- Evidence preservation that stands up in court
- Weekly board-level security reports

2.6 Business Model (As Designed)

LBRO is designed as a **SaaS platform** with the following commercial layers:

1. **SDK Integration** — Developers embed the Python or Node.js SDK into their applications. The SDK sends security events to LBRO automatically. This creates a sticky integration.
2. **Per-Project Billing** — Each project represents one application or service. Companies with many apps pay for multiple projects.
3. **Platform Administration** — The `super_admin` role manages the entire platform — can create organizations, assign admins to projects, view audit logs across everything.
4. **Report Downloads** — Weekly PDF security reports and compliance audit PDFs are generated on-demand. These are what CEOs and boards actually want to see.

2.7 Value Proposition

> "LBRO tells you what happened, proves it in court, and tells you what law requires you to do about it — automatically."

2.8 Real-World Use Case

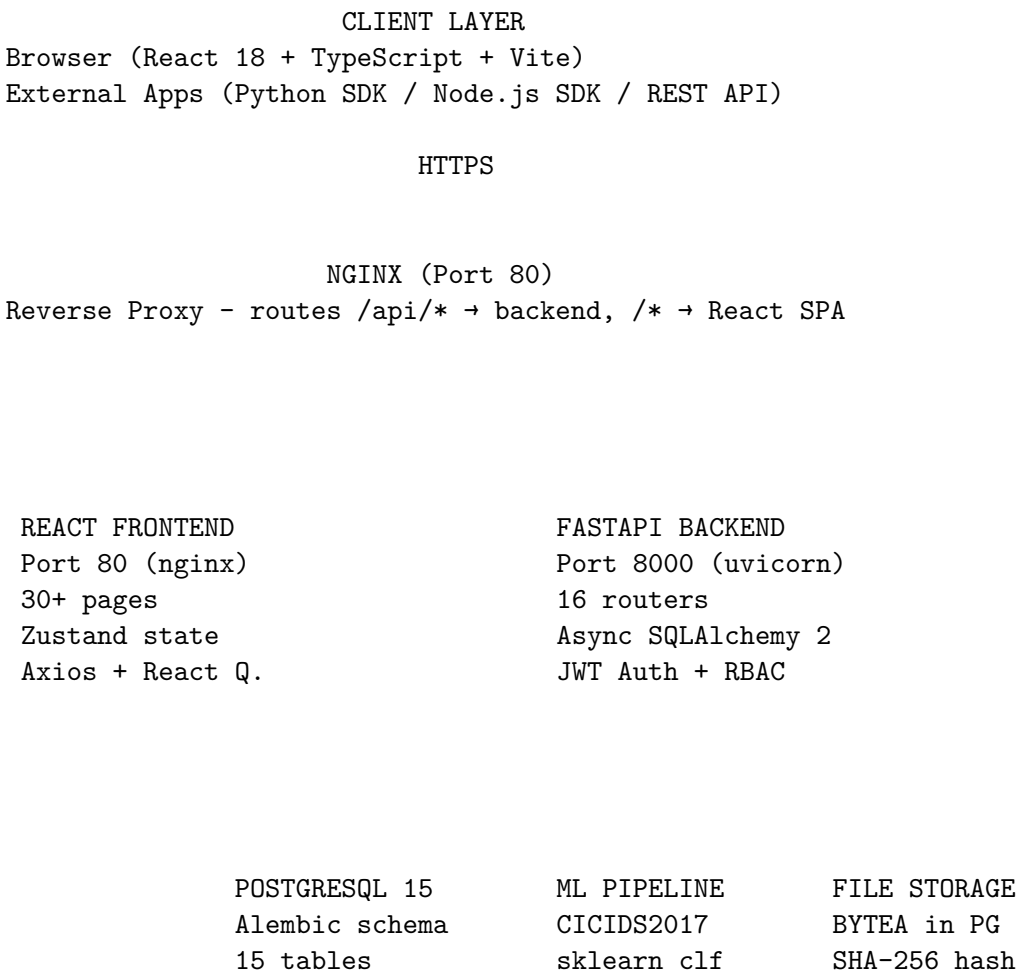
Scenario: A healthcare startup uses LBRO.

1. Their Node.js backend detects unusual login attempts at 3 AM.
2. The LBRO Node.js SDK automatically sends the event to LBRO as an incident.
3. LBRO's ML pipeline classifies it as `SSH-Patator` (brute force on SSH), severity: HIGH.
4. Because health data is involved, LBRO automatically creates a HIPAA compliance obligation with a 60-day notification deadline.
5. An analyst logs in at 9 AM, sees the incident already classified, uploads a server log as forensic evidence.
6. LBRO records exactly who uploaded the evidence, when, from what IP address — the chain of custody.
7. At end of week, the CISO downloads the PDF security report and shares it with the board.

Total manual work: 10 minutes. Without LBRO: 3 days of coordination across 5 different tools.

3 SECTION 2 — SYSTEM ARCHITECTURE

3.1 Architecture Overview



3.2 Component Breakdown

3.2.1 2.1 Frontend

What it is: A React 18 single-page application (SPA) with TypeScript, built with Vite.

Key pages (all source-verified): - Login, Register, ForgotPassword - Dashboard (security score, incident summary) - Incidents list + filtering - Incident Detail (full investigation workspace with notes, timeline, evidence, IOC, PDF report) - Evidence Vault - Compliance Audit - Weekly Reports - Live Events (SSE stream) - Project Setup Wizard - Integrations (SDK code snippets) - API Documentation - Settings (profile, demo data generation) - Infrastructure Health - ML Metrics - Threat Intelligence - Users Management (admin only) - Audit Logs

Key libraries: - `zustand` — global auth state management - `@tanstack/react-query` — server

state, caching, background refresh - `axios` — HTTP client with JWT interceptor - `recharts` — charts on dashboard - `lucide-react` — icons

Authentication flow in frontend: - Access token stored in **module-level memory variable** (not Zustand store, not localStorage) — survives re-renders, wiped on page refresh - Refresh token stored in **sessionStorage** (tab-scoped, wiped when tab closes) - On page reload: `onRehydrateStorage` detects sessionStorage refresh token → silently calls `/auth/refresh` → restores access token in memory - 401 on any API call → try silent refresh once → if fails, logout + redirect to `/login`

3.2.2 2.2 Backend

What it is: A FastAPI application running on Python 3.12 with asyncio throughout.

Middleware stack (outermost → innermost): 1. `SecurityHeadersMiddleware` — adds X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, HSTS 2. `RateLimitMiddleware` — in-memory per-IP rate limiting (60 req/min default) 3. `TrustedHostMiddleware` — blocks requests with spoofed Host headers 4. `CORSMiddleware` — configured CORS origins from `CORS_ORIGINS` env var 5. Request context middleware — assigns X-Request-ID, measures response time

16 Registered Routers:

Router	Prefix	Purpose
auth	/api/v1/auth	Login, register, refresh, profile, API key rotation
incidents	/api/v1/incidents	CRUD + investigation workspace
evidence	/api/v1/evidence	Upload, download, verify integrity
notifications	/api/v1/notifications	Regulatory deadline notifications
compliance	/api/v1/compliance	Obligations, assessments, score
users	/api/v1/users	User management (admin only)
ml	/api/v1/ml	Classify, flows, metrics
dashboard	/api/v1/dashboard	Stats, charts, security score
audit	/api/v1/audit	Audit log query
infrastructure	/api/v1/infrastructure	Health status
security_score	/api/v1/security	Score computation
reports	/api/v1/reports	Weekly JSON + PDF, compliance PDF
projects	/api/v1/projects	Project CRUD
demo	/api/v1/demo	Generate sample data
platform	/api/v1/platform	Super-admin platform endpoints
events	/api/v1/events	SSE live event stream

3.2.3 2.3 Database

What it is: PostgreSQL 15 accessed via SQLAlchemy 2 with async support (asyncpg driver).

15 Core Tables:

```
users
  projects (owner_id → users.id)
      incidents (project_id → projects.id)
          evidence (incident_id → incidents.id)
```

```
            chain_of_custody (evidence_id → evidence.id)
         compliance_records (incident_id → incidents.id)
         investigation_notes (incident_id → incidents.id)
         notifications (incident_id → incidents.id)
         incident_actions (incident_id → incidents.id)
      compliance_obligations (project_id → projects.id)
      compliance_assessments (project_id → projects.id)
  audit_logs (user_id → users.id)
  revoked_tokens (standalone - stores revoked JTI strings)
  project_members (user_id × project_id)
  security_events (standalone - raw security event log)
```

3.2.4 2.4 ML Pipeline

What it is: A scikit-learn classifier trained on the CICIDS2017 network intrusion detection dataset.

Flow:

```
Network flow features (78 CICIDS2017 columns)
    ↓
Feature vector assembly
    ↓
StandardScaler (if scaler.pkl available)
    ↓
Classifier.predict_proba() → probability for each class
    ↓
argmax → attack_category + confidence score
    ↓
If confidence < 0.75 → needs_analyst_review = True
    ↓
Top-10 feature importance (by absolute value)
    ↓
Stored in incident.attack_category, incident.confidence_score
```

Fallback: If model file is missing or input has fewer than 10 non-zero features, a rule-based heuristic runs instead (checks port 21 = FTP, port 22 = SSH, packet rate > 10,000 = DDoS, etc.).

3.2.5 2.5 Evidence Vault

What it is: Files are stored as binary data (BYTEA) directly in PostgreSQL. No S3 dependency.

Upload flow: 1. File size checked (100 MB limit) 2. Content-type checked against allowlist 3. First bytes checked for dangerous signatures (PE/ELF executables, shell shebangs, PHP) 4. Filename sanitized — `re.sub(r'[^\w.\-]', '_', name)[:255]` 5. SHA-256 hash computed 6. Binary stored in `evidence.file_data` (deferred column — not loaded on list queries) 7. Chain-of-custody record created: action=`uploaded`, user, IP, timestamp

Download flow: 1. Ownership verified (incident belongs to user's project, or admin) 2. `file_data` loaded (deferred column — only fetched now) 3. Hash re-verified against `sha256_hash` 4. Chain-of-custody record: action=`accessed` 5. `Response` object returned (not `StreamingResponse`, to avoid binary corruption)

3.2.6 2.6 Compliance Engine

What it is: Three-table system in PostgreSQL that tracks regulatory obligations.

- `compliance_records` — Incident-linked obligations. When an incident involves personal data, LBRO auto-creates records with calculated deadlines (GDPR 72h, HIPAA 60 days, DPDPA 72h).
- `compliance_obligations` — Per-project control checklist. Analysts check off controls like "GDPR Article 32 — encryption at rest." Replaces the old localStorage approach.
- `compliance_assessments` — Point-in-time snapshot of compliance score, stored so you can see how your posture changed over time.

Score formula (from reports.py):

```
score = compliant_controls / total_controls × 100
```

Security score formula (from reports.py):

```
score = 100
score -= min(open_critical × 15, 45)
score -= min(open_high × 8, 24)
score -= min((open_medium + open_low) × 2, 10)
score -= min(users_without_mfa × 4, 20)
score -= 10  (if >50 403s in last 24h)
score -= min(overdue_compliance × 5, 15)
score += 5   (if all users have MFA)
score += 5   (if 100% compliance met)
```

Grade: A (90), B (75), C (60), D (40), F (<40)

3.2.7 2.7 Authentication

What it is: JWT HS256 with jti (JWT ID) revocation.

Token design: - Access token: 30 minutes, contains `sub` (user_id), `jti` (uuid4), `role`, `email`, `permissions[]` - Refresh token: 7 days, contains `sub`, `jti`, `type=refresh` - On logout: jti stored in `revoked_tokens` table — checked on every authenticated request

3.2.8 2.8 RBAC

What it is: Four roles with 30+ permissions defined in a single `ROLE_PERMISSIONS` dictionary.

```
super_admin → ALL permissions (platform + project)
    ↓
admin → ALL permissions (project scope only)
    ↓
analyst → viewer permissions + CREATE/UPDATE incidents, UPLOAD evidence,
          MANAGE compliance, GENERATE reports, VIEW audit, etc.
    ↓
viewer → READ incidents, DOWNLOAD evidence, VIEW dashboard,
         READ notifications, VIEW compliance/reports/ML
```

Every 403 is written to the `audit_logs` table. Every super_admin bypass is logged with `action="super_admin_access"`.

3.2.9 2.9 Project Isolation (IDOR Prevention)

What it is: Non-privileged users only see incidents in projects they own.

Implementation:

```python
def _owner_id_for(user):
    # Admin and super_admin see everything (returns None = no filter)
    if user.role in ("admin", "super_admin"):
        return None
    # Viewer and analyst only see their own projects
    return user.id
```

In `IncidentService._ownership_scope()`, if `owner_id` is set, a JOIN to `projects` filters by `projects.owner_id = owner_id`. This applies to ALL incident list/get endpoints.

3.2.10 2.10 Docker Infrastructure

Local (`docker-compose.yml`): - `postgres:16-alpine` — database with health check - `localstack:3.4` — simulates S3, SQS, SecretsManager locally - `api` — FastAPI backend (2 uvicorn workers, uvloop) - `frontend` — Nginx serving React SPA

Production (`docker-compose.prod.yml`): - Same stack without LocalStack - `lbro-prod-network` bridge — API port 8000 NOT exposed to host network - All traffic goes through Nginx on port 80 - `lbro-postgres-data` persistent volume

4 SECTION 3 — TECH STACK DEEP DIVE

4.1 3.1 React 18

What it is: A JavaScript library for building user interfaces using components.

Why we used it: React 18 introduced concurrent rendering, which makes the UI more responsive when many things update at once (e.g., live events stream + dashboard updates). Its component model and large ecosystem (React Query, Zustand, Recharts) made it the most practical choice.

Advantages: Huge ecosystem, great tooling (Vite, DevTools), unidirectional data flow is easy to reason about, JSX is readable.

Disadvantages: Not a full framework — routing, state management, data fetching all require separate libraries.

Alternatives: Angular (full framework, more opinionated, better for large enterprise teams), Vue.js (simpler API, smaller bundle), SvelteKit (compiles away the framework, great performance).

Why not Angular? Angular's steep learning curve and two-way binding model add complexity that isn't needed for a focused security dashboard. React's composition model matches the page-by-page structure of LBRO better.

4.2 3.2 TypeScript

What it is: A superset of JavaScript that adds static type checking.

Why we used it: With 30+ pages, 15+ API functions, and complex data models (incidents, evidence, compliance), TypeScript catches type errors at compile time instead of runtime. It also makes the codebase self-documenting — when you read `Incident.severity`, TypeScript tells you it can only be `"critical" | "high" | "medium" | "low" | "info"`.

Advantages: Catches bugs early, better IDE autocomplete, mandatory for large codebases.

Disadvantages: Adds compilation step, verbose for simple scripts, `any` type can escape the type system.

Alternatives: Plain JavaScript (faster to write, worse to maintain), Flow (Facebook's type checker, less popular).

4.3 3.3 Vite

What it is: A modern frontend build tool that serves files natively as ES modules during development and bundles with Rollup for production.

Why we used it: Vite's dev server starts in milliseconds (no bundling on start). React + TypeScript with Webpack can take 30+ seconds to start. Vite uses native browser ES module support for instant hot-module replacement (HMR).

Advantages: Near-instant dev startup, fast HMR, built-in TypeScript support, excellent React plugin.

Disadvantages: Production build uses Rollup (different from dev, so occasional build-only bugs).

Alternatives: webpack (powerful but slow), Create React App (deprecated), Parcel (zero-config).

In LBRO specifically: Vite's dev proxy (`proxy: { '/api': 'http://localhost:8000' }`) forwards API calls to the FastAPI backend, so the frontend and backend can run on different ports without CORS issues in development.

4.4 3.4 FastAPI

What it is: A modern Python web framework for building REST APIs with automatic OpenAPI documentation and native async/await support.

Why we used it: FastAPI is built on Pydantic and Starlette. Every endpoint's request body and response is a Pydantic model, so validation is automatic. FastAPI generates a Swagger UI at `/docs` (in debug mode) and OpenAPI JSON at `/openapi.json` — no extra work needed.

Advantages: - Automatic request validation via Pydantic - Auto-generated OpenAPI docs - Native async — handles many concurrent requests without threads - Dependency injection system (used for auth, DB sessions, permissions) - Type hints everywhere → self-documenting code

Disadvantages: Smaller community than Django/Flask, async requires thinking carefully about blocking code.

Alternatives: Flask (simpler, synchronous, larger community), Django REST Framework (batteries-included, ORM, admin panel), Express.js (Node.js).

Why not Flask? Flask is synchronous. For a security platform with many concurrent users, async is essential. FastAPI's dependency injection system is also much cleaner for auth middleware than Flask decorators.

4.5 3.5 Python 3.12

What it is: The programming language running the entire backend.

Why we used it: Python 3.12 is the fastest CPython release to date. More importantly, Python's scikit-learn ecosystem for ML is unmatched — no other language has the same depth of ML tooling. Having the ML pipeline and the API in the same language eliminates a service boundary.

Advantages: ML ecosystem (scikit-learn, numpy, joblib), readable, asyncio native, pydantic.

Disadvantages: Slower than Go/Rust for CPU-bound work (mitigated with asyncio.to_thread for ML inference).

Alternatives: Go (much faster, no ML ecosystem), Node.js (JavaScript everywhere, weaker ML), Java/Spring Boot (enterprise, verbose).

4.6 3.6 PostgreSQL 15

What it is: A production-grade relational database with support for JSON, UUID, binary data (BYTEA), and strong ACID guarantees.

Why we used it: - **ACID guarantees** — When a critical security incident is recorded, you cannot have partial writes. Either the entire record is written or nothing is. - **BYTEA** — Stores binary evidence files directly in the database, eliminating the S3 dependency for small deployments.

- **UUID primary keys** — PostgreSQL's native `UUID` type makes IDs non-sequential (attackers can't enumerate records by incrementing an integer). - **JSON columns** — `network_features`, `containment_actions`, `affected_jurisdictions` are stored as JSON, allowing flexible schema for fields that vary per incident type.

Advantages: ACID, complex queries, joins, indexes, full-text search, extensions.

Disadvantages: Vertical scaling limits, more complex to shard horizontally than MongoDB.

Alternatives: MongoDB (schema-flexible, horizontal scaling, but no ACID by default), MySQL (less feature-rich), SQLite (file-based, great for tests).

Why not MongoDB? Security incident data has strong relationships: incident → evidence → chain_of_custody → user. These are best expressed as foreign key relationships with referential integrity. MongoDB's document model would require denormalization or application-level joins, both of which are error-prone.

4.7 3.7 SQLAlchemy 2 (Async)

What it is: Python's most powerful ORM (Object-Relational Mapper), using the new SQLAlchemy 2 API with full async support via `AsyncSession`.

Why we used it: SQLAlchemy 2's `mapped_column` and `Mapped[T]` syntax makes models fully typed. With asyncpg as the driver, database queries are truly non-blocking.

Key pattern in LBRO:

```
async with AsyncSession(engine) as session:
    result = await session.execute(
        select(Incident).where(Incident.project_id == project_id)
    )
    incidents = result.scalars().all()
```

`pool_pre_ping=True` — Before using a connection from the pool, SQLAlchemy sends a simple ping query. If the database restarted, the dead connection is discarded and a fresh one is used. This prevents "connection reset" errors in production.

4.8 3.8 Docker & Docker Compose

What it is: Docker containers package an application and all its dependencies into an isolated, reproducible unit. Docker Compose orchestrates multiple containers.

Why we used it: - **Environment parity** — The code running on a developer's MacBook runs identically on the EC2 server. No "works on my machine" bugs. - **Dependency isolation** — PostgreSQL, Python 3.12, Node.js all run in separate containers with their own dependency graphs. - **Production deployment** — `docker compose -f docker-compose.prod.yml up -d` brings up the entire stack in one command.

Advantages: Reproducible builds, easy scaling (replicate containers), simple CI/CD.

Disadvantages: Overhead for simple applications, debugging inside containers requires extra steps, image layers grow over time.

Alternatives: Kubernetes (production-grade orchestration at scale, but overkill for v1), bare metal (fastest, but no isolation), Heroku/Render (PaaS, but less control).

In LBRO production: The API container is NOT exposed on the host network. Nginx is the only public entry point. This means you cannot directly reach the API on port 8000 from outside the Docker network — it's protected behind Nginx.

4.9 3.9 JWT (JSON Web Tokens)

What it is: A compact, signed token format used for authentication.

Structure: `header.payload.signature` - Header: `{"alg": "HS256", "typ": "JWT"}` - Payload: `{"sub": "user_uuid", "jti": "unique_id", "role": "analyst", "permissions": [...], "exp": timestamp}` - Signature: HMAC-SHA256 of header+payload using the SECRET_KEY

Why we used it: JWTs are stateless — the server doesn't need to look up a session in a database on every request. The token itself contains the user's role and permissions, so `require_permission()` can check authorization without a DB query.

Why jti revocation? The main weakness of JWTs is that they can't be invalidated before expiry. We fix this by storing revoked JTIs in the `revoked_tokens` table. On logout, the JTI is stored. Every authenticated request checks the JTI against this table. This adds one DB lookup per request but makes tokens properly revocable.

Advantages: Stateless, self-contained, cross-domain friendly.

Disadvantages: Cannot invalidate without extra DB lookup, tokens carry sensitive data (role, email) visible to anyone who decodes the base64 payload.

Alternatives: Session tokens (server-side session storage, simpler revocation), OAuth 2.0 (for third-party auth).

4.10 3.10 bcrypt (via passlib)

What it is: A password hashing algorithm specifically designed to be slow, making brute-force attacks expensive.

Why bcrypt and not SHA-256? SHA-256 is fast — a modern GPU can compute 10 billion SHA-256 hashes per second. bcrypt is designed to be slow (cost factor 12 in LBRO = 2^12 = 4096 rounds). The same GPU can only compute ~2000 bcrypt hashes per second. If the database is stolen, attackers cannot crack passwords efficiently.

Timing attack protection: LBRO computes `_DUMMY_HASH = hash_password("sentinel")` at module load time. When a login request comes in for an email that doesn't exist, LBRO still calls `verify_password(submitted, _DUMMY_HASH)` before returning 401. Without this, the missing-user path returns in ~1ms (no bcrypt work), while the wrong-password path returns in ~300ms (bcrypt ran). This ~299ms difference allows attackers to enumerate valid email addresses.

4.11 3.11 AWS EC2, S3, SQS

EC2 — What it is: Elastic Compute Cloud — a virtual machine in AWS. LBRO runs on a single EC2 instance in `ap-south-1` (Mumbai region).

S3 — What it is: Simple Storage Service — object storage. In LBRO, the schema supports S3 for evidence storage (legacy fields `s3_key`, `s3_bucket` exist in the `evidence` table) but the current implementation stores evidence as BYTEA in PostgreSQL. S3 fields are kept for future migration.

SQS — What it is: Simple Queue Service — a managed message queue. LBRO's incident router optionally enqueues incident events to an SQS queue when `SQS_QUEUE_URL` is configured. This allows downstream systems (data warehouses, alerting systems) to consume security events asynchronously without blocking the API response.

Why AWS? AWS is the dominant cloud platform. EC2 is the simplest entry point — one virtual machine, one SSH key, one `docker compose up`. SQS provides reliable, at-least-once delivery for incident events with zero infrastructure to manage.

Alternatives: GCP (Cloud Run, Pub/Sub), Azure (VMs, Service Bus), self-hosted (Kafka for queuing, bare metal servers).

4.12 3.12 Pydantic

What it is: A Python data validation library that validates data at runtime using Python type hints.

Why we used it: Every request body in LBRO is a Pydantic model. When a client sends `{"severity": "extreme"}`, Pydantic immediately rejects it because `severity` must be one of `"critical" | "high" | "medium" | "low" | "info"`. FastAPI integrates with Pydantic so this validation happens automatically before the endpoint function runs.

```
class IncidentCreate(BaseModel):
    title: str
    severity: Literal["critical", "high", "medium", "low", "info"] = "medium"
    project_id: Optional[UUID] = None
```

Advantages: Runtime validation, auto-generated JSON schema, data coercion (string "true" → bool True).

4.13 3.13 Alembic

What it is: A database migration tool for SQLAlchemy. It tracks database schema changes as numbered Python scripts.

Why we used it: Every time we add a column, create a table, or add an index, we write an Alembic migration. This creates a versioned history of the database schema that can be applied (upgrade) or rolled back (downgrade) reliably.

Why `NullPool` in migrations? Standard SQLAlchemy connection pools maintain long-lived connections. Alembic migrations need to start, apply changes, and exit cleanly. `NullPool` creates a new connection for each operation and closes it immediately — no connection leaks.

Migration chain in LBRO: 12 numbered versions, from the initial schema through the platform layer (super_admin, ProjectMember, SecurityEvent).

4.14 3.14 Axios + React Query

Axios — What it is: An HTTP client for the browser and Node.js.

In LBRO: `apiClient` is a configured Axios instance with: - `baseURL` set to `API_BASE_URL` (from environment, defaults to `''` for relative URLs via Vite proxy) - Request interceptor: attaches `Authorization: Bearer <token>` to every request - Response interceptor: on 401, tries silent token refresh → if that fails, logs out

React Query (TanStack Query) — What it is: A server-state management library. It handles caching, background refetching, and loading/error states for API calls.

Why not Redux? Redux is for client state (UI state, user preferences). React Query is for server state (data from the API). Mixing them creates complexity. LBRO uses Zustand for auth state (client) and React Query for everything from the server.

4.15 3.15 scikit-learn, NumPy, Joblib

scikit-learn: The ML framework. Provides the `predict_proba()` method used for multi-class probability output.

NumPy: Numerical array operations. Feature vectors are `np.ndarray` of `float32`. `np.argmax()` finds the highest-confidence class.

Joblib: Used for model serialization (alternative to pickle for sklearn models). The model is loaded from a `.pkl` file at startup.

SHA-256: Used for evidence integrity. `hashlib.sha256(file_bytes).hexdigest()` produces a 64-character hex string stored in `evidence.sha256_hash`. Before any download, the hash is recomputed and compared. If they don't match, the evidence has been tampered with.

5 SECTION 4 — END-TO-END SYSTEM FLOW

5.1 Complete Example: SQL Injection Detected

Let us trace exactly what happens when a web application detects a SQL injection attempt and reports it to LBRO.

5.1.1 Step 1 — Application Detects the Event

```python
# In customer's Python web application
from lbro_sdk import LBROClient

client = LBROClient(
    project_api_key="proj_abc123...",
    base_url="https://lbro.mycompany.com"
)

client.report_event({
    "title": "SQL Injection in login form",
    "severity": "critical",
    "attack_category": "Web Attack - Sql Injection",
    "source_ip": "192.168.1.100",
    "destination_port": 5432,
    "personal_data_involved": True
})
```

5.1.2 Step 2 — Authentication at the API Gateway

The SDK includes the project API key in the `X-API-Key` header.

In `dependencies.py`:

```python
async def get_project_from_api_key(x_api_key: str = Header(...)):
    # Look up the Project whose api_key matches
    project = await db.execute(
        select(Project).where(Project.api_key == x_api_key)
    )
    if not project:
        raise HTTPException(401)
    return project
```

Note on BUG-7: This comparison is plaintext (not hashed). A database read would expose all project API keys. Deferred to v1.1.

5.1.3 Step 3 — Incident Created in FastAPI

The `POST /api/v1/incidents` endpoint:

1. Validates the request body via Pydantic (`IncidentCreate` schema)

2. Creates an `Incident` ORM object with UUID primary key
3. Assigns `status = "new"`, records `detected_at = now()`
4. If `personal_data_involved = True`, marks `personal_data_involved = True`

5.1.4 Step 4 — ML Classification

```
# In incidents router, after creating the incident
if incident.network_features:
    result = await asyncio.to_thread(
        classifier.predict, incident.network_features
    )
    incident.attack_category = result["attack_category"]
    incident.confidence_score = result["confidence"]
    incident.needs_analyst_review = result["needs_review"]
    incident.ml_model_version = result["model_version"]
```

Why `asyncio.to_thread`? ML inference (scikit-learn) is CPU-bound, not I/O-bound. Running it on the asyncio event loop would block all other requests. `asyncio.to_thread` moves it to a thread pool worker.

Classification result:

```
{
  "attack_category": "Web Attack - Sql Injection",
  "confidence": 0.87,
  "severity": "critical",
  "needs_review": false,
  "top_features": [
    {"feature": "destination_port", "value": 5432.0, "importance": 0.42},
    {"feature": "flow_packets_per_sec", "value": 210.0, "importance": 0.18}
  ]
}
```

5.1.5 Step 5 — Compliance Obligations Auto-Generated

Because `personal_data_involved = True`:

```
# GDPR: 72-hour notification deadline
ComplianceRecord(
    incident_id=incident.id,
    regulation="GDPR",
    obligation="Notify supervisory authority within 72 hours",
    deadline=incident.detected_at + timedelta(hours=72),
    is_met=False
)

# DPDPA (India): 72-hour deadline
ComplianceRecord(
    incident_id=incident.id,
    regulation="DPDPA",
```

```
    obligation="Report personal data breach within 72 hours",
    deadline=incident.detected_at + timedelta(hours=72),
    is_met=False
)
```

5.1.6 Step 6 — Analyst Investigates (Frontend)

An analyst logs in, navigates to the Incident Detail page:

1. **Timeline tab** — Shows `incident.created` action at timestamp, all status changes
2. **Evidence tab** — Upload network captures, screenshots, log files
3. **Notes tab** — Write investigation notes (only they or an admin can edit/delete their notes)
4. **IOC tab** — View source IP, destination port, protocol from the incident
5. **Related tab** — ML-suggested related incidents with same attack category

5.1.7 Step 7 — Evidence Uploaded

```
Analyst uploads: network_capture.pcap (2.4 MB)
    ↓
Content-type check: application/octet-stream
Dangerous signature check: PCAP header, not PE/ELF
SHA-256: a3f2b1c4d5e6...
Binary stored in evidence.file_data (BYTEA)
Chain-of-custody: action=uploaded, by=alice@company.com, ip=10.0.0.5
```

5.1.8 Step 8 — Incident Closed

Analyst updates status to `closed`. A `IncidentAction` record is created:

```
action_type: "status_change"
description: "Status changed from investigating to closed"
performed_by: analyst_user_id
automated: false
```

5.1.9 Step 9 — Compliance Marked Met

Analyst confirms they sent the GDPR notification. They call `POST /compliance/records/{id}/mark-met`. The `ComplianceRecord.is_met = True`, `met_at = now()`.

5.1.10 Step 10 — Weekly Report Generated

Every Monday (or on demand), the admin downloads the PDF:

```
GET /api/v1/reports/weekly/pdf
    ↓
_build_report_data() queries DB for:
  - Open incidents by severity
  - New this week, closed this week
  - Top 5 attack categories
  - Most targeted ports
  - Evidence count
```

```
  - Compliance met/total
  - Users without MFA
  - Recent 403 spike count
    ↓
Security score computed (starts at 100, penalized for each risk)
    ↓
reportlab generates PDF
    ↓
StreamingResponse with Content-Disposition: attachment
```

5.1.11 Data Flow Summary

```
External App
      X-API-Key header

FastAPI /api/v1/incidents (POST)
      Pydantic validation

AuthService validates API key
      Project found

Incident created in PostgreSQL
      network_features JSON

ML Classifier (asyncio.to_thread)
      attack_category, confidence

Compliance records auto-created
      deadlines calculated

AuditLog written (action=incident.created)


201 Created response → SDK → Customer app
      (async)

Analyst opens frontend → investigates → closes


Weekly PDF report generated by CEO/board
```

6 SECTION 5 — SECURITY DESIGN

6.1 5.1 Authentication

Mechanism: JWT HS256 (HMAC-SHA256).

Token contents:

```
{
  "sub": "550e8400-e29b-41d4-a716-446655440000",
  "jti": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "role": "analyst",
  "email": "bob@company.com",
  "permissions": ["incident:create", "incident:read", "evidence:upload"],
  "exp": 1722600000,
  "iat": 1722598200,
  "type": "access"
}
```

Why embed permissions in the token? The `require_permission()` dependency can check authorization without a database query. The permissions list is regenerated on every token refresh, so role changes take effect within 30 minutes.

Access token lifetime: 30 minutes. Short enough that stolen tokens expire quickly.

Refresh token lifetime: 7 days. Stored in browser sessionStorage (not localStorage — wiped when the browser tab closes).

JTI Revocation: Every authenticated request:

```
revoked = await db.execute(
    select(RevokedToken).where(RevokedToken.jti == token_jti)
)
if revoked:
    raise HTTPException(401, "Token has been revoked")
```

6.2 5.2 Authorization (RBAC)

Single source of truth: `ROLE_PERMISSIONS` dictionary in `rbac.py`.

Permission check pattern:

```
@router.post("/incidents")
async def create_incident(
    current_user: Annotated[User, Depends(require_permission(Permission.CREATE_INCIDENT))]
):
    ...
```

`require_permission` is a FastAPI dependency factory. It returns a dependency that: 1. Calls `get_current_active_user` to validate the JWT 2. Checks `has_permission(user.role, permission)` 3. If fails: writes an AuditLog entry with `action="permission_denied"`, raises 403

Every 403 is permanently recorded in audit_logs with: - user_id, user_email - requested permission - IP address, user_agent - request path, method

6.3 5.3 Project Isolation

Problem: Without isolation, a viewer account at Company A could read Company B's incidents if they guessed the UUID. This is called an **Insecure Direct Object Reference (IDOR)** vulnerability.

Solution:

```
def _owner_id_for(user) -> Optional[UUID]:
    if user.role in ("admin", "super_admin"):
        return None  # No filter - sees everything
    return user.id   # Filter by this user's projects

# In queries:
owner_id = _owner_id_for(current_user)
if owner_id is not None:
    query = query.join(Project).where(Project.owner_id == owner_id)
```

This is applied to: - `GET /api/v1/incidents` (list) - `GET /api/v1/incidents/{id}` (single) - `GET /api/v1/evidence/{id}` (single evidence) - `GET /api/v1/evidence/{id}/download` - Evidence listing for an incident - Compliance records - Dashboard statistics

Test coverage: 11 dedicated IDOR tests in `test_project_isolation.py`.

6.4 5.4 API Keys

Two types:

1. **User API Keys** — `lbro_` + 32 random bytes (URL-safe base64). Generated via `POST /auth/api-key/rotate`. Used for authenticating as a specific user from scripts.
2. **Project API Keys** — `proj_` + 32 random bytes. Generated when a project is created. Used by external applications to submit incidents to that specific project.

Known limitation (BUG-7): Both types stored as plaintext in the database. If an attacker gains database read access, all API keys are exposed without further cracking. Fix: hash with SHA-256 on storage, compare `sha256(submitted_key)` at auth time. Deferred to v1.1.

6.5 5.5 Password Security

Hashing: bcrypt via passlib, cost factor 12 (4096 rounds).

Timing attack defence: `_DUMMY_HASH` precomputed at module load. Login always calls `verify_password`, even when the email doesn't exist, to prevent user enumeration via response latency.

Account lockout: After `MAX_LOGIN_ATTEMPTS` (5) consecutive failures, `locked_until = now() + 15 minutes`. The locked_until check runs BEFORE bcrypt verification, so locked accounts fail fast without wasting CPU on bcrypt.

Password validation (frontend): RegisterPage enforces: - Minimum 8 characters - At least one uppercase letter - At least one number
- At least one symbol (e.g., !, @, #)

6.6 5.6 Evidence Integrity (SHA-256 + Chain of Custody)

SHA-256 hashing:

```
sha256_hash = hashlib.sha256(file_bytes).hexdigest()
# Stored: "a3f2b1c4d5e6f7a8..."  (64 hex chars)
```

On download, the hash is recomputed:

```
if hashlib.sha256(evidence.file_data).hexdigest() != evidence.sha256_hash:
    raise HTTPException(500, "Evidence integrity check failed")
```

Chain of custody (`chain_of_custody` table): Every interaction with evidence creates an immutable record:

action	performed_by	ip_address	timestamp
uploaded	alice@co.com	10.0.0.5	2026-08-01T09:15:00Z
accessed	bob@co.com	10.0.0.8	2026-08-01T14:22:00Z
verified	carol@co.com	10.0.0.9	2026-08-02T10:00:00Z

The `hash_at_time` column records the SHA-256 at the moment of each access — so if the hash ever changes, you can pinpoint exactly when.

6.7 5.7 Audit Logs

Every significant action is recorded in `audit_logs`: - All 403 permission denials - All super_admin bypasses - User role changes (old_role → new_role) - Incident create/update/delete - Evidence upload/download

Query endpoint: `GET /api/v1/audit/logs` — analyst and above can view.

6.8 5.8 Security Headers

`SecurityHeadersMiddleware` adds these headers to every response:

Header	Value	Purpose
X-Content-Type-Options	nosniff	Prevents MIME-type sniffing
X-Frame-Options	DENY	Prevents clickjacking
X-XSS-Protection	1; mode=block	Legacy XSS filter
Strict-Transport-Security	max-age=31536000	Forces HTTPS
Referrer-Policy	strict-origin-when-cross-origin	Controls referrer header

6.9 5.9 Rate Limiting

Implementation: In-memory dictionary keyed by client IP. Counter resets every 60 seconds. Default limit: 60 requests per minute.

Limitation: Not Redis-backed. In a multi-worker deployment (LBRO uses 2 uvicorn workers), each worker has its own counter. An attacker can make N × 60 = 120 requests per minute by hitting different workers. For true rate limiting across workers, Redis would be needed.

Response on limit: HTTP 429 with `Retry-After` header.

6.10 5.10 CORS Configuration

`CORSMiddleware` is configured with origins from the `CORS_ORIGINS` environment variable. The variable supports three formats: - Comma-separated: `http://localhost:3000,http://localhost:5173` - JSON array: `["http://localhost:3000"]` - Single origin: `http://13.203.164.225`

Why this matters: Without CORS configuration, a malicious website cannot make authenticated API calls on behalf of a logged-in LBRO user (the browser will block it). CORS is enforced at the browser level.

6.11 5.11 Current Security Limitations

Limitation	Impact	Planned Fix
Plaintext API keys (BUG-7)	DB read exposes all keys	v1.1: SHA-256 hashing
In-memory rate limiting	Multi-worker bypass	v1.1: Redis-backed
No refresh token rotation	Stolen refresh token reusable until expiry	v1.1
MFA fields exist but not enforced	mfa_enabled column in DB, TOTP not implemented	v1.1
Email change without verification	Session hijack risk	v1.1

7 SECTION 6 — DATABASE DESIGN

7.1 6.1 Why PostgreSQL?

Requirement	PostgreSQL Feature
Store binary evidence files	BYTEA column type
Non-guessable primary keys	Native UUID type
Flexible per-incident metadata	JSON/JSONB columns
ACID compliance for audit trail	Full ACID transactions
Async Python access	asyncpg driver
Free, production-grade	Open source

7.2 6.2 Full Schema

7.2.1 users

```
id            UUID        PK
email         VARCHAR(255) UNIQUE
username      VARCHAR(100) UNIQUE
full_name     VARCHAR(255)
hashed_password VARCHAR(255)
role          VARCHAR(50) [viewer|analyst|admin|super_admin]
is_active     BOOLEAN
is_verified   BOOLEAN
mfa_enabled   BOOLEAN
mfa_secret    VARCHAR(100) nullable
last_login    TIMESTAMPTZ
failed_login_attempts INTEGER
locked_until  TIMESTAMPTZ nullable
api_key       VARCHAR(128) UNIQUE nullable
```

7.2.2 projects

```
id            UUID        PK
name          VARCHAR(200)
slug          VARCHAR(120) UNIQUE INDEX
description   TEXT
environment   VARCHAR(50) [development|staging|production]
status        VARCHAR(50) [active|archived]
owner_id      UUID        FK→users.id SET NULL
api_key       VARCHAR(64) UNIQUE INDEX  ("proj_" prefix)
created_at    TIMESTAMPTZ
updated_at    TIMESTAMPTZ
```

7.2.3 incidents

```
id                UUID        PK
external_id       VARCHAR(100) UNIQUE INDEX  (for idempotency)
title             VARCHAR(500)
description       TEXT
status            VARCHAR(50) INDEX  [new|triaging|contained|eradicating|recovering|closed|reo
severity          VARCHAR(50) INDEX  [critical|high|medium|low|info]
attack_category   VARCHAR(100)  (ML output)
confidence_score  FLOAT
ml_model_version  VARCHAR(50)
needs_analyst_review BOOLEAN
source_ip         VARCHAR(45)
destination_ip    VARCHAR(45)
source_port       INTEGER
destination_port  INTEGER
protocol          VARCHAR(20)
network_features  JSON      (78 CICIDS2017 features)
containment_actions JSON
containment_completed_at TIMESTAMPTZ
affected_jurisdictions JSON
personal_data_involved BOOLEAN
health_data_involved   BOOLEAN
project_id        UUID  FK→projects.id SET NULL INDEX
assigned_to       UUID  FK→users.id SET NULL INDEX
created_by        UUID  FK→users.id SET NULL INDEX
detected_at       TIMESTAMPTZ
closed_at         TIMESTAMPTZ
created_at        TIMESTAMPTZ
updated_at        TIMESTAMPTZ
```

7.2.4 evidence

```
id                UUID        PK
incident_id       UUID        FK→incidents.id CASCADE INDEX
filename          VARCHAR(500)
original_filename VARCHAR(500)
content_type      VARCHAR(200)
file_size         INTEGER
s3_key            TEXT  nullable  (legacy field)
s3_bucket         VARCHAR(255) nullable  (legacy field)
sha256_hash       VARCHAR(64)
file_data         BYTEA  DEFERRED  (loaded only on download)
description       TEXT
tags              TEXT  (JSON array as text)
is_immutable      BOOLEAN DEFAULT TRUE
uploaded_by       UUID  FK→users.id SET NULL
created_at        TIMESTAMPTZ
```

7.2.5 chain_of_custody

```
id                UUID        PK
evidence_id       UUID        FK→evidence.id CASCADE INDEX
action            VARCHAR(100)  [uploaded|accessed|exported|verified]
performed_by      UUID  FK→users.id SET NULL
performed_by_name VARCHAR(255)
ip_address        VARCHAR(45)
user_agent        TEXT
notes             TEXT
hash_at_time      VARCHAR(64)  (SHA-256 at time of action)
created_at        TIMESTAMPTZ
```

7.2.6 compliance_records

```
id            UUID        PK
incident_id   UUID        FK→incidents.id CASCADE INDEX
regulation    VARCHAR(50) [GDPR|HIPAA|DPDPA]
jurisdiction  VARCHAR(100)
obligation    VARCHAR(500)
deadline      TIMESTAMPTZ
is_met        BOOLEAN DEFAULT FALSE
met_at        TIMESTAMPTZ nullable
notes         TEXT
created_at    TIMESTAMPTZ
updated_at    TIMESTAMPTZ
```

7.2.7 compliance_obligations

```
id                UUID        PK
project_id        UUID        FK→projects.id CASCADE INDEX
framework         VARCHAR(50) [GDPR|HIPAA|DPDPA|PCI-DSS|SOC2]
control_id        VARCHAR(100)  (e.g., "Art-32", "164.312(a)")
control_name      VARCHAR(255)
description       TEXT
status            VARCHAR(50) [not_started|in_progress|compliant|non_compliant|not_applicable]
evidence_reference TEXT
score             FLOAT DEFAULT 0.0
recommendations   TEXT
last_updated      TIMESTAMPTZ
created_at        TIMESTAMPTZ
```

7.2.8 compliance_assessments

```
id                UUID        PK
project_id        UUID        FK→projects.id CASCADE INDEX
framework         VARCHAR(50)
overall_score     FLOAT
total_controls    INTEGER
```

```
compliant_controls INTEGER
assessment_date   TIMESTAMPTZ
notes             TEXT
created_at        TIMESTAMPTZ
```

7.2.9 audit_logs

```
id              UUID        PK
user_id         UUID  FK→users.id SET NULL INDEX
project_id      UUID  FK→projects.id SET NULL INDEX
user_email      VARCHAR(255)
action          VARCHAR(200) INDEX
resource_type   VARCHAR(100) INDEX
resource_id     VARCHAR(36)
ip_address      VARCHAR(45)
user_agent      TEXT
request_method  VARCHAR(10)
request_path    TEXT
response_status INTEGER
details         JSON
created_at      TIMESTAMPTZ INDEX
```

7.2.10 investigation_notes

```
id            UUID        PK
incident_id   UUID        FK→incidents.id CASCADE INDEX
author_id     UUID        FK→users.id SET NULL INDEX
content       TEXT
created_at    TIMESTAMPTZ
updated_at    TIMESTAMPTZ
```

7.2.11 revoked_tokens

```
id            UUID        PK
jti           VARCHAR(36) UNIQUE INDEX
user_id       UUID  nullable
revoked_at    TIMESTAMPTZ
expires_at    TIMESTAMPTZ
```

7.3 6.3 Key Design Decisions

UUIDs as primary keys: - Random UUIDs are non-sequential. An attacker cannot enumerate records by trying `id=1, id=2, id=3`. - PostgreSQL's native `UUID` type stores them efficiently as 16 bytes (not as a string). - SQLAlchemy generates the UUID in Python (`default=uuid.uuid4`) before the INSERT, so the application always knows the ID immediately.

Deferred BYTEA column:

```
file_data: Mapped[bytes | None] = deferred(mapped_column(LargeBinary, nullable=True))
```

`deferred` means this column is NOT included in `SELECT *` queries. It's only loaded when explicitly accessed. Without this, listing 50 evidence records would load 50 files (potentially gigabytes) from the database just to show filenames.

CASCADE vs SET NULL: - `ON DELETE CASCADE` — evidence is deleted when its incident is deleted (evidence has no meaning without an incident) - `ON DELETE SET NULL` — `assigned_to` becomes NULL when a user is deleted (the incident still exists, just unassigned) - `ON DELETE SET NULL` — `project_id` becomes NULL when a project is deleted (the incident is preserved as an orphan)

Indexes: - `incidents.status` — frequent filter: `WHERE status IN ('new', 'triaging', ...)` - `incidents.severity` — frequent filter: `WHERE severity = 'critical'` - `incidents.project_id` — used in every scoped query for project isolation - `projects.slug` — used for human-readable project URLs - `audit_logs.created_at` — used for time-range queries - `revoked_tokens.jti` — used on every authenticated request

7.4 6.4 Alembic Migrations

What Alembic does:

```
Initial schema (001_initial)
    ↓
Add indexes (002_indexes)
    ↓
Add compliance tables (003_compliance)
    ↓
Add evidence vault (004_evidence)
    ↓
Add investigation notes (005_notes)
    ↓
Add project API keys (006_project_api_keys)
    ↓
... (up to 012_platform_layer)
```

Each migration file has `upgrade()` and `downgrade()` functions. To apply all: `alembic upgrade head`. To roll back one version: `alembic downgrade -1`.

Why not `create_all()`? SQLAlchemy's `Base.metadata.create_all()` creates tables if they don't exist but cannot ALTER tables (add columns, change types, add indexes). Alembic can do all of that safely in production without downtime.

8 SECTION 7 — MACHINE LEARNING PIPELINE

8.1 7.1 The Dataset: CICIDS2017

What it is: The Canadian Institute for Cybersecurity Intrusion Detection System 2017 dataset. It is the most widely-used benchmark dataset for network intrusion detection.

What it contains: 2.8 million network flow records, captured over 5 days, with 78 features extracted from packet captures using CICFlowMeter. Each record is labelled with either `BENIGN` or one of 14 specific attack types.

Why this dataset? - It's realistic — captured on a real network, not simulated - It's labelled by domain experts - It's publicly available for research - It covers the most common attacks companies actually face

Dataset characteristics: - 78 input features (network flow statistics) - 15 output classes (including BENIGN) - Significant class imbalance — most traffic is BENIGN

8.2 7.2 The 78 CICIDS2017 Features

These are network flow statistics — not raw packet data. A "flow" is a group of packets between the same source and destination.

Categories of features:

Category	Features	What They Measure
Basic flow	destination_port, flow_duration	Port and duration of the connection
Packet counts	total_fwd_packets, total_bwd_packets	How many packets sent each direction
Packet sizes	fwd_packet_length_max/min/mean/std, bwd_packet_length_*	Statistical distribution of packet sizes
Flow rates	flow_bytes_per_sec, flow_packets_per_sec	Throughput metrics
Inter-arrival times	flow_iat_mean/std/max/min	Timing between consecutive packets
Flag counts	fin_flag_count, syn_flag_count, rst_flag_count, etc.	TCP control flags
Bulk statistics	fwd_avg_bytes_per_bulk, bwd_avg_bulk_rate	Burst behaviour
Subflows	subflow_fwd_packets, subflow_bwd_packets	Subdivided flow statistics
Active/Idle	active_mean/std/max/min, idle_mean/std	Active vs idle periods

Why 78 features? More features give the model more signal. A DDoS attack has a very high `flow_packets_per_sec`. An SSH brute force has a high `syn_flag_count` on port 22. A web attack

targets port 80 or 443. These statistics capture attack patterns that raw packet analysis would miss.

8.3 7.3 The 15 Attack Classes

Class	Severity	What It Is
BENIGN	info	Normal traffic — no attack
DoS Hulk	critical	HTTP flood that exhausts server resources
PortScan	medium	Systematic scan to find open ports
DDoS	critical	Distributed flood from multiple sources
DoS GoldenEye	high	HTTP POST/GET flood using GoldenEye tool
FTP-Patator	high	Brute force against FTP (port 21)
SSH-Patator	high	Brute force against SSH (port 22)
DoS slowloris	high	Slow HTTP attack — keeps connections open
DoS Slowhttptest	high	Similar to slowloris, different tool
Bot	critical	Malware-controlled automated traffic
Web Attack - Brute Force	high	HTTP brute force on web applications
Web Attack - XSS	medium	Cross-site scripting payloads
Infiltration	critical	Network infiltration via backdoor
Web Attack - Sql Injection	critical	SQL injection attempts
Heartbleed	critical	OpenSSL Heartbleed buffer overread exploit

8.4 7.4 Model Architecture

The classifier is a scikit-learn pipeline loaded from `cicids2017_classifier.pkl`. The code's handling reveals it is designed around a probabilistic classifier with `predict_proba()` output (returns class probabilities, not just a class label).

Code comments specifically reference **GaussianNB** (Gaussian Naive Bayes) behaviour on sparse inputs: > "GaussianNB collapses to PortScan (confidence=1.0) when the input vector is sparse"

This is why LBRO includes a sparse-input guard: if fewer than 10 of the 78 features are non-zero, the heuristic fallback runs instead of the model.

Optional scaler: If `scaler.pkl` exists, features are scaled via `StandardScaler.transform()` before prediction. StandardScaler subtracts the mean and divides by standard deviation, so all features have mean=0 and std=1. This is critical for distance-based algorithms.

8.5 7.5 Prediction Flow

```
def predict(self, features: dict) -> dict:
    # 1. Assemble feature vector (78 values in canonical CICIDS2017 order)
    vec = [float(features.get(f) or 0.0) for f in CICIDS2017_FEATURES]
    vec = np.array(vec, dtype=np.float32).reshape(1, -1)  # shape: (1, 78)

    # 2. Sparse input guard
    if np.count_nonzero(vec) < 10:
        return self._heuristic_predict(features)
```

```
    # 3. Apply scaler if available
    if self._scaler:
        vec = self._scaler.transform(vec)

    # 4. Predict probabilities for all 15 classes
    probas = self._model.predict_proba(vec)[0]  # shape: (15,)

    # 5. Pick highest probability class
    class_idx = np.argmax(probas)
    confidence = float(probas[class_idx])
    attack_category = ATTACK_CLASSES[class_idx]

    # 6. Needs review if below threshold
    needs_review = confidence < 0.75  # settings.ML_CONFIDENCE_THRESHOLD

    # 7. Top features by absolute value (explainability)
    top_features = sorted by abs(vec[i]), top 10

    return {
        "attack_category": attack_category,
        "confidence": confidence,
        "severity": SEVERITY_MAP[attack_category],
        "needs_review": needs_review,
        "probabilities": {class: prob for each class},
        "top_features": [...],
        "model_version": "1.0.0"
    }
```

8.6 7.6 Heuristic Fallback

When the model is unavailable or input is too sparse, a rule-based heuristic runs:

```
if pkt_rate > 10000:    → "DDoS"
elif syn_flags > 1000:  → "DoS Hulk"
elif dst_port == 21:    → "FTP-Patator"
elif dst_port == 22:    → "SSH-Patator"
elif dst_port in (80, 443, 8080): → "Web Attack - Brute Force"
else:                   → "BENIGN"
```

Heuristic confidence is always `0.65` (below the 0.75 threshold), so `needs_review` is always `True` for heuristic results — flagging them for human review.

8.7 7.7 Explainability

Top features computation:

```
# Sort feature indices by their absolute value (highest first)
sorted_idx = np.argsort(np.abs(vec))[::-1][:10]
```

```
# Compute relative importance
importance[i] = abs(vec[i]) / sum(abs(vec))
```

This is a simple but interpretable approach. It tells the analyst: "These 10 features contributed most to the classification." For example, a classification of `SSH-Patator` might show `syn_flag_count=1500` as the top feature.

This is not SHAP values. It's a simplified proxy. SHAP (SHapley Additive exPlanations) would be more accurate but is much slower to compute in real-time. A future improvement.

8.8 7.8 ML API Endpoints

```
GET /api/v1/ml/flows     - Returns recent classified incidents as CICIDS flow records
GET /api/v1/ml/metrics   - Returns feature importance, per-class confidence, tactic distributi
POST /api/v1/ml/classify - Classify a raw feature dict (for testing/SDK use)
```

8.9 7.9 Limitations

Limitation	Impact
CICIDS2017 is from 2017	Newer attack types (e.g., Log4Shell, supply chain) not in training data
No online learning	Model cannot update itself from new incidents without retraining
GaussianNB on sparse input	Degrades to heuristic, which is less accurate
No GPU acceleration	CPU-only inference (acceptable for current scale)
Explainability is simplified	Top features by absolute value, not SHAP

9 SECTION 8 — COMPLIANCE ENGINE

9.1 8.1 Overview

The compliance engine has two modes:

1. **Incident-linked compliance** (`compliance_records` table) — Auto-generated obligations when an incident involves personal or health data. Has hard deadlines (72h, 60 days).
2. **Project-level compliance controls** (`compliance_obligations` table) — Manual checklist for frameworks like GDPR, HIPAA, DPDPA. Analysts check off controls as they implement them.

9.2 8.2 The Three Regulations

9.2.1 GDPR (General Data Protection Regulation — EU)

Aspect	Detail
Who it applies to	Any company handling EU citizens' personal data
Breach notification deadline	72 hours to notify supervisory authority
Trigger in LBRO	`personal_data_involved = True` on incident
Example obligation	"Notify supervisory authority within 72 hours of becoming aware"

GDPR is the most important regulation for European companies and any company with EU customers. Fines can reach 4% of global annual revenue or €20M, whichever is higher.

9.2.2 HIPAA (Health Insurance Portability and Accountability Act — USA)

Aspect	Detail
Who it applies to	US healthcare providers, insurers, and their business associates
Breach notification	Notify HHS within 60 days; notify affected individuals without unreasonable delay
Trigger in LBRO	`health_data_involved = True` on incident
Example obligation	"Notify HHS of breach within 60 days"

9.2.3 DPDPA (Digital Personal Data Protection Act — India)

Aspect	Detail
Who it applies to	Any company processing digital personal data of Indian residents
Breach notification	72 hours to notify the Data Protection Board
Trigger in LBRO	`personal_data_involved = True` + Indian jurisdiction
Example obligation	"Report personal data breach to Data Protection Board within 72 hours"

9.3 8.3 How Obligations Are Auto-Generated

In `incidents.py`, when creating an incident:

```
if incident.personal_data_involved:
    # GDPR - 72 hours
    await compliance_svc.create_record(
        incident_id=incident.id,
        regulation="GDPR",
        jurisdiction="EU",
        obligation="Notify supervisory authority within 72 hours",
        deadline=incident.detected_at + timedelta(hours=72)
    )
    # DPDPA - 72 hours
    await compliance_svc.create_record(
        regulation="DPDPA",
        deadline=incident.detected_at + timedelta(hours=72)
    )

if incident.health_data_involved:
    # HIPAA - 60 days (converted to hours)
    await compliance_svc.create_record(
        regulation="HIPAA",
        deadline=incident.detected_at + timedelta(hours=1440)
    )
```

9.4 8.4 Compliance Score Calculation

From `compliance_service.py`:

`score = (compliant_controls / total_controls) × 100`

Example: - Project has 10 GDPR controls - 7 are marked "compliant" - Score = $7/10 \times 100 = 70\%$

The `ComplianceAssessment` table stores point-in-time snapshots of this score, so you can see how it changes over time.

9.5 8.5 PDF Report Generation

`GET /api/v1/reports/compliance/pdf` generates a professional PDF using **reportlab**:

1. Header: LBRO logo, report date, overall compliance %
2. Summary table: Total / Met / Overdue / Pending / Compliance %
3. Per-regulation breakdown (GDPR section, HIPAA section, etc.)
4. Each obligation: green (MET), red (OVERDUE), amber (PENDING)
5. Footer: "Generated by LBRO on [date]. Confidential."

9.6 8.6 Compliance Dashboard API

```
GET /api/v1/compliance/dashboard  - Overall posture summary
GET /api/v1/compliance/obligations  - All controls for a project/framework
POST /api/v1/compliance/obligations  - Upsert a control status
PATCH /api/v1/compliance/obligations/{id}  - Update status/evidence
GET /api/v1/compliance/score  - Live compliance score
POST /api/v1/compliance/assess  - Take a snapshot assessment
GET /api/v1/compliance/assessments  - Historical snapshots
POST /api/v1/compliance/records/{id}/mark-met  - Mark a deadline obligation as fulfilled
```

10 SECTION 9 — BUSINESS MODEL & SAAS ROADMAP

10.1 9.1 Who Would Buy LBRO?

Customer Type	Why They Need It
SaaS startups	Need compliance tracking (GDPR) but can't afford a dedicated CISO
Healthcare technology companies	HIPAA breach notification is a legal requirement, not optional
Indian tech companies	DPDPA compliance as new Indian privacy law takes effect
Financial technology (fintech)	PCI-DSS compliance for card data; regulatory audits
Mid-market enterprises	Have security events but no structured incident response process
Security operations centres (SOCs)	Replace multiple spreadsheet tools with one platform

10.2 9.2 How Integration Works (SDK Model)

Developer workflow:

```
# Install: pip install lbro-sdk
from lbro_sdk import LBROClient

client = LBROClient(
    project_api_key=os.environ["LBRO_PROJECT_KEY"],
    base_url="https://lbro.yourcompany.com"
)

# Report an event automatically
client.report_event({
    "title": "Unusual login from new country",
    "severity": "high",
    "attack_category": "Infiltration",
    "source_ip": request.remote_addr,
    "personal_data_involved": True
})
```

The SDK makes LBRO an **integration platform**, not just a dashboard. Once a developer installs the SDK, LBRO gets a continuous stream of security events from their application. This creates strong user stickiness.

10.3 9.3 Multi-Project Support (Already Implemented)

Each company can have multiple projects (e.g., one for their main application, one for their admin portal, one for their mobile API). Each project has: - Its own API key - Its own incidents, evidence, compliance - Its own team members (viewer, analyst, admin can be assigned per project — foundation exists via `project_members` table)

10.4 9.4 Platform Administration (Already Implemented)

The `super_admin` role can: - View all data across all projects (`PLATFORM_VIEW_ALL` permission) - Manage any user (`PLATFORM_MANAGE_USERS`) - Create/archive any project (`PLATFORM_MANAGE_PROJECTS`) - View all audit logs (`PLATFORM_VIEW_AUDIT`) - Check system health (`PLATFORM_SYSTEM_HEALTH`) - Assign roles across the platform (`PLATFORM_ASSIGN_ROLES`)

This role is designed for the **LBRO platform operator** — the company running LBRO as a service for their customers.

10.5 9.5 Potential SaaS Roadmap

Phase 1 (Current — v1.0): - Single deployment, multi-project - SDK integration (Python + Node.js) - Self-hosted on customer's EC2

Phase 2 (v1.1): - Organisation layer above projects (for large enterprises with business units) - Redis-backed rate limiting (multi-worker safe) - Hashed API keys - TOTP MFA implementation - Refresh token rotation

Phase 3 (v2.0 — SaaS): - LBRO runs as a managed service - Customers get sub-domains (`company.lbro.io`) - Per-incident or per-project pricing - SSO/SAML integration - Webhook delivery for real-time alerting - Advanced ML with online learning

10.6 9.6 The LBRO Agent Vision

Based on the SDK architecture in the repository, a future "LBRO Agent" would: - Monitor application logs automatically (no SDK calls needed) - Auto-classify events before sending to LBRO - Run as a sidecar container alongside customer applications - Support 17 SDK languages (stubs exist in the repository for Python, Node.js, Go, Java, Ruby, PHP, Rust, C#, Swift, Kotlin, Dart, Elixir, Scala, Haskell, R, C++, C)

11 SECTION 10 — ENGINEERING DECISIONS & TRADE-OFFS

11.1 10.1 FastAPI vs Flask

Aspect	FastAPI	Flask
Async support	Native (asyncio)	Limited (via Quart or gevent)
Request validation	Automatic (Pydantic)	Manual
API documentation	Auto-generated (OpenAPI)	Manual
Type hints	First-class	Optional
Learning curve	Moderate	Lower
Community size	Growing fast	Large, mature

Decision: FastAPI.

For LBRO, async was not optional. The application handles many concurrent requests (SSE live events, multiple analysts working simultaneously, dashboard auto-refresh). Flask's synchronous model would require multiple threads or processes, adding complexity.

Pydantic validation means we cannot receive an invalid incident severity value — it's rejected before the endpoint function runs. With Flask, we would write validation code manually.

11.2 10.2 PostgreSQL vs MongoDB

Aspect	PostgreSQL	MongoDB
ACID transactions	Full ACID	Document-level only (ACID with replica sets from v4.0)
Relationships	Foreign keys, JOINs	Application-level, no foreign keys
Schema flexibility	Strict (with JSON escape hatch)	Dynamic
Complex queries	Excellent (SQL)	Good (aggregation pipeline)
Binary storage	BYTEA column	GridFS (separate)
Read performance at scale	Needs indexes	Horizontal sharding native

Decision: PostgreSQL.

LBRO data is highly relational: incident → project → user, evidence → incident → chain_of_custody. A graph of 15 tables with foreign key constraints ensures referential integrity. You cannot accidentally delete a project and leave orphaned incidents with dangling references.

The `BYTEA` column for evidence storage is a PostgreSQL-specific feature. MongoDB would require GridFS, adding complexity.

The `JSON` columns in `incidents.network_features` give us the schema flexibility of MongoDB where we need it, within a relational database.

11.3 10.3 React vs Angular

Aspect	React	Angular
Learning curve	Moderate	Steep
Bundle size	Small (library)	Larger (full framework)
State management	External (Zustand, Redux)	Built-in (Services, RxJS)
Data binding	One-way	Two-way option
TypeScript	Optional (but natural)	Mandatory
Opinionation	Low	High

Decision: React.

LBRO's frontend is a purpose-built security dashboard, not a general enterprise CRUD application. React's lightweight composition model is better suited — each page is a focused component with its own data requirements (React Query), not a complex form with two-way bindings.

Vite + React starts in under 1 second and hot-reloads in milliseconds. Angular CLI with Webpack takes 15-30 seconds to start. Developer productivity matters.

11.4 10.4 JWT vs Session Tokens

Aspect	JWT	Session Tokens
Server state	None (stateless)	Server must store session
Revocation	Extra DB lookup (jti)	Delete session from store
Cross-domain	Easy	Requires cookie sharing
Token size	Larger (contains data)	Small (just a random ID)
Performance	No DB lookup per request*	Session lookup required

*Except for jti revocation check.

Decision: JWT with jti revocation.

LBRO embeds the user's role and permissions in the token. The `require_permission()` dependency can authorize a request without hitting the database. For a security platform where auth happens on every API call, reducing DB lookups improves performance.

The jti revocation table adds one DB lookup per request (SELECT WHERE jti = ?) but gives us the ability to invalidate tokens immediately — essential for a security platform where compromised accounts must be locked out instantly.

11.5 10.5 asyncio.to_thread for ML Inference

Problem: scikit-learn inference is CPU-bound. Running it on the asyncio event loop would block ALL other requests for the duration of inference (~50-200ms per prediction).

Solution:

```
result = await asyncio.to_thread(classifier.predict, features)
```

`asyncio.to_thread` submits the CPU work to the default ThreadPoolExecutor. The event loop continues handling other requests while the ML inference runs in a separate thread. When inference completes, the result is returned to the awaiting coroutine.

Alternative considered: A separate ML microservice. More complex (another service to deploy, network latency, serialization overhead), but would allow the ML service to scale independently. Not justified for v1.0 scale.

11.6 10.6 BYTEA vs S3 for Evidence

Aspect	BYTEA (PostgreSQL)	S3
Setup complexity	None	AWS credentials, bucket policies
Cost	Server storage cost	Per-GB storage + per-request
Scalability	Vertical (bigger DB)	Virtually unlimited
Availability	DB must be up	Independent of app
Backup	Included in DB backup	Separate S3 versioning
100MB limit	Enforced in code	Can be lifted

Decision: BYTEA for v1.0.

For a v1.0 product, the simplest solution that works is correct. S3 adds AWS dependency, IAM configuration, bucket policies, and pre-signed URL management. A startup's first security deployment should not fail because someone forgot to configure an S3 bucket policy.

The `s3_key` and `s3_bucket` fields are preserved in the schema for a future migration to S3. When LBRO has paying customers uploading gigabytes of evidence daily, migrating to S3 becomes justified.

11.7 10.7 In-Memory Rate Limiting vs Redis

Aspect	In-Memory	Redis
Setup	None	Redis instance required
Multi-worker safe	No	Yes
Persistent across restarts	No	Configurable
Performance	Fastest (no network)	Very fast
Cost	None	Redis instance cost

Decision: In-memory for v1.0.

LBRO runs 2 uvicorn workers. With in-memory rate limiting, each worker independently tracks request counts. An attacker can send $2 \times 60 = 120$ requests per minute by hitting both workers. This is an accepted trade-off for v1.0. Redis is in `requirements.txt` and the `REDIS_URL` config exists — the infrastructure for upgrading is already prepared.

---

12 SECTION 11 — CHALLENGES & HOW WE SOLVED THEM

12.1 Challenge 1: User Enumeration via Login Timing

The Problem:

The original login code was:

```
user = await db.get_user_by_email(email)
if not user:
    raise HTTPException(401)  # Returns immediately (~1ms)

if not verify_password(password, user.hashed_password):
    raise HTTPException(401)  # Returns after bcrypt (~300ms)
```

An attacker can send 1000 login requests and measure response times. Emails that return in 1ms = not in the database. Emails that return in 300ms = registered account. This leaks your entire user list.

The Fix:

```
_DUMMY_HASH: str = hash_password("lbro-dummy-constant-time-sentinel")

async def login(data):
    user = await db.get_user_by_email(data.email)
    # ALWAYS call verify_password - even if user not found
    password_ok = verify_password(
        data.password,
        user.hashed_password if user else _DUMMY_HASH
    )
    if not user or not password_ok:
        raise HTTPException(401)
```

Now both paths take ~300ms. Timing attack eliminated.

Key learning: Even logically correct code can leak information through side channels. Always think about what information response TIMING reveals.

12.2 Challenge 2: Investigation Note Authorization Gap

The Problem:

The original `update_investigation_note` endpoint checked that the incident exists and the user has `UPDATE_INCIDENT` permission — but did NOT check that the user authored the note. Any analyst could edit any other analyst's investigation notes.

The Fix:

```
can_edit = (
    note.author_id == current_user.id
```

```
    or current_user.role == Role.ADMIN.value
    or is_super_admin(current_user.role)
)
if not can_edit:
    raise HTTPException(403, "You can only edit your own notes")
```

Key learning: "Has permission to do X" is not the same as "is authorized to do X on this specific resource." Always check both.

12.3 Challenge 3: Evidence IDOR (Insecure Direct Object Reference)

The Problem:

The original `list_evidence` and `get_evidence_by_id` endpoints did not verify that the requesting user owns the project containing the incident. A viewer from one organization could request `GET /evidence/{uuid}` and receive another organization's forensic files.

The Fix: Applied the same `_ownership_scope()` pattern used in incident queries to evidence queries. The JOIN to `incidents → projects → owner_id` enforces that only evidence in the user's projects is accessible.

Key learning: Every resource that belongs to a project must independently enforce project ownership. "The incident is scoped" does not automatically scope the incident's children.

12.4 Challenge 4: Test Fixture Design vs Project Isolation

The Problem:

A test for "viewer can list incidents" used `carol` (a viewer) expecting to see incidents created by `alice` (an admin). The test assumed viewers see all incidents. But project isolation correctly limits viewers to their own projects. Carol owned no projects, so she saw zero incidents. The test was wrong, not the code.

The Fix: Added `carol_project` (created directly in DB) and `carol_viewable_incident` (incident in Carol's project, created by Alice). The test now correctly verifies that Carol CAN see incidents in her OWN project.

Key learning: Test fixtures must match the authorization model. Write tests that prove the CORRECT behaviour, not tests that assume incorrect behaviour.

12.5 Challenge 5: bcrypt Hash Truncation Bug

The Problem:

An intermediate attempt at the timing fix used:

```
_DUMMY_HASH = "$2b$12$eImiTXuWVxfM37uY4JANjQ"
```

This is a bcrypt "config string" — it has the prefix and salt but no checksum. Passlib rejected it with `MissingDigestError: expected bcrypt hash, got bcrypt config string instead`.

The Fix: Compute the hash properly:

```
_DUMMY_HASH: str = hash_password("lbro-dummy-constant-time-sentinel")
```

Key learning: Never hardcode partial cryptographic values. Use the library to generate correct values.

12.6 Challenge 6: Docker Nginx DNS Cache (502 Gateway)

The Problem:

On the EC2 server, nginx started before the FastAPI API container was ready. Nginx resolved the hostname `api` (Docker internal DNS) to an IP that later changed. All requests returned `502 Bad Gateway` even after the API was running.

Diagnosis clue: `curl http://localhost:8000/health` returned 200 (API was up), but `curl http://localhost/health` returned 502 (nginx couldn't reach it).

The Fix: Restart the nginx/frontend container:

```
docker compose restart frontend
```

This forces nginx to re-resolve `api` DNS, picking up the correct IP.

Key learning: Docker's internal DNS resolves container names at connection time, not at startup. If a container restarts and gets a new IP, nginx's upstream cache can become stale.

12.7 Challenge 7: Postgres Password URL-Encoding

The Problem:

The PostgreSQL password contained `@` (`teamvkdk@99`). In the DATABASE_URL:

```
postgresql+asyncpg://lbro:teamvkdk@99@postgres:5432/lbro
```

SQLAlchemy's URL parser sees two `@` characters. It interprets the last `@` as the host separator, so it tries to connect to host `99@postgres` (wrong) with password `teamvkdk` (incomplete). Intermittent DNS errors resulted.

The Fix: URL-encode the `@` as `%40`:

```
postgresql+asyncpg://lbro:teamvkdk%4099@postgres:5432/lbro
```

Key learning: Database passwords in URLs must have special characters URL-encoded. Use `urllib.parse.quote(password, safe='')` to encode.

12.8 Challenge 8: ML GaussianNB Sparse Input Collapse

The Problem:

When an incident is created manually (no network capture, no CICIDS2017 flow data), the feature vector is all zeros except for a few manually filled fields. GaussianNB — which assumes Gaussian probability distributions per feature — gives maximum confidence to whatever the dominant class is when all features are zero (it defaults to `PortScan` with confidence=1.0).

The Fix: Count non-zero features:

```
if np.count_nonzero(vec) < 10:
    return self._heuristic_predict(features)  # Rule-based fallback
```

Key learning: ML models must be validated on the actual distribution of inputs they will receive in production, not just on the training dataset.

13 SECTION 12 — INTERVIEW PREPARATION (50 Q&A)

13.1 Architecture Questions

Q1. Describe the LBRO architecture in one minute. A. LBRO has three layers. The frontend is React 18 with TypeScript, served by Nginx, which also reverse-proxies API calls to FastAPI. The backend is FastAPI with 16 routers, running on Python 3.12 with async SQLAlchemy against PostgreSQL 15. The ML pipeline is a scikit-learn classifier trained on CICIDS2017 that runs on each new incident via asyncio.to_thread. Everything runs in Docker containers orchestrated by Docker Compose.

Q2. Why did you choose FastAPI over Django or Flask? A. FastAPI is async-native, which matters for a security platform handling concurrent requests. It provides automatic Pydantic validation — invalid requests are rejected before the endpoint runs. It also auto-generates OpenAPI documentation. Flask is synchronous and requires manual validation. Django would have added unnecessary ORM complexity since we're using SQLAlchemy directly.

Q3. How does data flow from an external application to LBRO's dashboard? A. The external app sends a POST to `/api/v1/incidents` with an `X-API-Key` header (project API key). FastAPI validates the key, creates the incident in PostgreSQL, runs ML classification via asyncio.to_thread, auto-generates compliance records, and writes to the audit log. The analyst then sees this incident on the dashboard, which polls the API via React Query.

Q4. What is SSE and where does LBRO use it? A. Server-Sent Events — a one-way HTTP push mechanism where the server streams events to the browser over a persistent connection. LBRO uses it for the Live Events page (`GET /api/v1/events/stream`). When new security events arrive, they're pushed to connected browsers in real-time without polling.

Q5. How does LBRO handle the N+1 query problem? A. SQLAlchemy's `lazy="selectin"` on relationships like `incident.actions` loads related records with a single IN query rather than one query per incident. The evidence `file_data` column uses `deferred` to avoid loading binary data when only listing evidence metadata.

13.2 Backend Questions

Q6. Walk me through what happens when a user calls POST /auth/login. A. The request body is validated by Pydantic (`LoginRequest`). The user is fetched by email. If the account is locked, 403 is returned before bcrypt. Then `verify_password` is called regardless of whether the user exists (timing attack protection). If password fails, `failed_login_attempts` is incremented. If it reaches 5, `locked_until` is set to 15 minutes from now. On success, an access token (30 min) and refresh token (7 days) are returned. The JWT payload includes the user's role and permissions.

Q7. What is jti and why is it important? A. `jti` is the JWT ID — a UUID4 embedded in every token. When a user logs out, the jti is stored in the `revoked_tokens` table. Every authenticated request checks if the jti is revoked. Without this, JWTs are valid until they expire — you can't log someone out. With jti, you can invalidate a token instantly.

Q8. Explain the RBAC implementation. A. Four roles: super_admin > admin > analyst >

viewer. All permissions are defined in `ROLE_PERMISSIONS` — a single dict in `rbac.py` that maps each role to a set of Permission enums. `require_permission(Permission.CREATE_INCIDENT)` is a FastAPI dependency factory. When used as an endpoint parameter, it validates the JWT, checks the role's permissions, logs every 403 to audit_logs, and raises HTTPException if denied.

Q9. What is project isolation and how is it enforced? A. Project isolation prevents viewers and analysts from accessing other users' incidents (IDOR prevention). `_owner_id_for(user)` returns `user.id` for non-admins, `None` for admins. If owner_id is set, every incident query JOINs to `projects` and filters by `projects.owner_id = owner_id`. This is applied in `IncidentService._ownership_scope()` and independently on evidence, compliance, and dashboard endpoints.

Q10. How does evidence integrity work? A. On upload, SHA-256 is computed over the file bytes: `hashlib.sha256(file_bytes).hexdigest()`. The 64-character hex is stored in `evidence.sha256_hash`. On download, the hash is recomputed from the stored BYTEA. If they don't match, the endpoint raises HTTP 500 and the download is aborted. The `chain_of_custody` table records every upload, access, and verification with timestamp, user, and IP.

Q11. Why is the evidence column deferred? A. The `file_data` column contains the raw binary of the uploaded file (up to 100MB). If it were loaded normally, listing 50 evidence records would pull up to 5GB from the database. `deferred` means SQLAlchemy omits this column from SELECT queries unless explicitly accessed (which only happens in the download endpoint).

Q12. Explain the middleware stack. A. Outermost (first to process requests): SecurityHeadersMiddleware adds security headers. RateLimitMiddleware checks per-IP request count. TrustedHostMiddleware validates the Host header to prevent host header injection. CORSMiddleware handles cross-origin requests. A manual request context middleware assigns X-Request-ID and measures response time.

Q13. What databases does LBRO use for tests vs production? A. Tests use SQLite in-memory (via `aiosqlite`). Production uses PostgreSQL 15 via asyncpg. The ORM (SQLAlchemy) abstracts the difference. `NullPool` is used in migrations and tests to avoid connection conflicts in async contexts.

Q14. How does LBRO prevent SQL injection? A. All database queries use SQLAlchemy's ORM or parameterized query builder. User input is never interpolated into SQL strings. For example: `select(Incident).where(Incident.id == incident_id)` — SQLAlchemy sends `WHERE id = ?` with the UUID as a bound parameter.

13.3 Frontend Questions

Q15. Why is the access token stored in module-level memory instead of localStorage? A. localStorage is accessible to JavaScript from any script on the page (XSS vulnerability). If an attacker injects malicious JavaScript, they can steal the token from localStorage. Module-level memory is only accessible to the application's own JavaScript module — XSS cannot reach it across module boundaries. The trade-off is the token is lost on page refresh, but the refresh token in sessionStorage handles silent re-authentication.

Q16. What is React Query and why is it used? A. TanStack Query (React Query) manages server state — data fetched from the API. It handles caching (so the same data isn't re-fetched if you navigate back to a page), background refetching (updates stale data without a full page reload),

and loading/error states. It reduces boilerplate compared to manual useEffect + useState patterns and prevents common bugs like stale closures.

Q17. How does the frontend handle token refresh? A. Axios has a response interceptor. On any 401 response to a non-auth endpoint, it tries silent refresh: `POST /auth/refresh` with the refresh token from sessionStorage. If the refresh succeeds, the new access token is stored in memory and the original request is retried. If the refresh fails (expired/revoked), the user is logged out and redirected to `/login`.

Q18. What is Vite's role in LBRO? A. Vite builds and serves the frontend. In development, it serves ES modules natively (instant HMR). In production, it bundles with Rollup into optimized chunks (`vendor.js` for React/Router, `charts.js` for Recharts). The dev server's proxy (`/api → localhost:8000`) routes API calls to FastAPI without CORS issues.

Q19. How does Zustand differ from Redux for state management? A. Zustand is minimal — a simple store with `set()` and `get()`. Redux requires actions, reducers, and middleware. Zustand's `persist` middleware handles sessionStorage sync automatically. For LBRO's auth state (user object, isAuthenticated flag, tokens), Zustand is perfectly sized. Redux would add significant boilerplate for minimal benefit.

13.4 Security Questions

Q20. What is a timing attack and how did you fix it in LBRO? A. A timing attack exploits differences in response time to infer information. In LBRO's login, if the user doesn't exist, the original code returned immediately (no bcrypt). If they do exist but the password is wrong, bcrypt runs (~300ms). An attacker measuring response times could identify which emails are registered. Fix: always run `verify_password`, even for nonexistent users, using a precomputed `_DUMMY_HASH`.

Q21. What is IDOR and where did it appear in LBRO? A. Insecure Direct Object Reference — when an API returns a resource based on its ID without checking if the requester owns it. In LBRO, the original evidence endpoints (`list_evidence`, `get_evidence_by_id`) did not verify project ownership. A viewer from Organization A could request evidence belonging to Organization B by guessing the UUID. Fixed by applying ownership JOINs to all evidence queries.

Q22. Explain bcrypt and why you chose it over SHA-256 for passwords. A. bcrypt is a password hashing function specifically designed to be slow. With cost factor 12, it performs 4096 rounds of key stretching. SHA-256 can compute 10 billion hashes/second on a GPU. bcrypt: ~2000/second. If the database is stolen, attackers cannot brute-force bcrypt-hashed passwords at scale. SHA-256 should never be used for passwords — it's a fast general-purpose hash.

Q23. What security headers does LBRO set and why? A. `X-Content-Type-Options: nosniff` — prevents browsers from interpreting response content differently from the declared MIME type (prevents certain XSS). `X-Frame-Options: DENY` — prevents LBRO pages from being embedded in iframes (prevents clickjacking). `Strict-Transport-Security` — forces HTTPS for 1 year. `Referrer-Policy` — limits referrer information sent to other sites.

Q24. What is CORS and how is it configured in LBRO? A. Cross-Origin Resource Sharing — a browser security policy that blocks cross-domain API calls unless the server explicitly allows them. Without CORS, a malicious website could make authenticated API calls on behalf of a logged-in LBRO user. LBRO configures `CORSMiddleware` with the `CORS_ORIGINS` env var. Only allowed origins can make API calls. `allow_credentials=True` enables cookies/Authorization headers.

Q25. What is the chain of custody and why is it legally important? A. The chain of custody documents who had access to evidence, when, and what they did with it. In a legal proceeding, digital evidence must demonstrate it has not been tampered with. LBRO's `chain_of_custody` table records every upload, access, and verification with user, IP address, timestamp, and the SHA-256 hash at that time. If the hash ever changes, you can prove exactly when and who was involved.

13.5 Database Questions

Q26. Why use UUIDs instead of auto-increment integers? A. Auto-increment IDs are sequential and predictable. An attacker can enumerate all incidents by trying ID 1, 2, 3… UUID4 values are random 128-bit numbers — guessing one has 1/2^122 probability. This is especially critical for a security platform where incident data is sensitive.

Q27. Explain ON DELETE CASCADE vs ON DELETE SET NULL. A. `CASCADE` — when the parent row is deleted, all child rows are deleted too. Used for evidence (no point keeping evidence without its incident). `SET NULL` — when the parent is deleted, the foreign key in the child becomes NULL. Used for `project_id` in incidents — if a project is deleted, the incidents are preserved as orphans. Used for `assigned_to` — if a user is deleted, the incident still exists, just unassigned.

Q28. What is Alembic and why not just use create_all()? A. Alembic tracks database schema changes as versioned migration scripts. `create_all()` creates tables if they don't exist but cannot ALTER tables — it can't add a column, change a type, or add an index to an existing table in production. Alembic can do all of this and can be rolled back with `downgrade()`.

Q29. What is pool_pre_ping and why is it set? A. When SQLAlchemy uses a connection from the pool, it normally assumes it's still alive. If the database restarted, the connection is dead. `pool_pre_ping=True` sends a cheap ping query (`SELECT 1`) before each connection is used. If the ping fails, the dead connection is discarded and a fresh one is opened. Prevents "broken pipe" errors after database restarts.

Q30. Why are compliance obligations stored in the database instead of localStorage? A. The original implementation stored checkbox states in browser localStorage. This means the compliance state is per-device and lost if the user clears their browser data. Moving it to the database means the state persists across devices, is shared between team members (an analyst can see what another analyst has checked off), and is included in backups.

13.6 ML Questions

Q31. What is CICIDS2017? A. The Canadian Institute for Cybersecurity Intrusion Detection System 2017 dataset. It contains 2.8 million network flow records captured over 5 days on a real network, labelled with 14 attack types + BENIGN. It's the most widely-used benchmark dataset for network intrusion detection research.

Q32. What is predict_proba() and why use it instead of predict()? A. `predict()` returns the single most likely class. `predict_proba()` returns the probability for EVERY class. LBRO uses `predict_proba()` because we need the confidence score (max probability) to decide if the classification needs human review. If max probability < 0.75, `needs_analyst_review = True`. This is not possible with `predict()` alone.

Q33. Why run ML inference with asyncio.to_thread? A. scikit-learn is CPU-bound —

it runs on the CPU and holds the GIL (Python's Global Interpreter Lock) during computation. Running it directly on the asyncio event loop would block the entire application for 50-200ms per prediction, making the API unresponsive to other requests. `asyncio.to_thread` offloads the CPU work to a thread pool, freeing the event loop.

Q34. What is a confidence threshold and how does LBRO use it? A. The confidence threshold is the minimum probability below which the model's prediction is considered unreliable. LBRO's threshold is 0.75 (75%). If the model's top probability is below 0.75, `needs_analyst_review = True` is set on the incident. This flags it for a human to verify — preventing incorrect automatic classifications from being acted upon without review.

Q35. What would you improve about the ML pipeline? A. 1. Upgrade to SHAP values for proper feature importance instead of absolute value sorting. 2. Add online learning — update the model weights as analysts confirm/correct classifications. 3. Add CICIDS-2023 or newer datasets to cover post-2017 attack types (Log4Shell, supply chain attacks). 4. Move to a separate ML microservice for independent scaling. 5. Add GPU inference for large-scale deployments.

13.7 Docker Questions

Q36. Explain the Docker network architecture in LBRO production. A. In production, all containers are on `lbro-prod-network` (bridge network). Nginx is the only container with port 80 exposed to the host. The FastAPI API container has port 8000 exposed only on the Docker internal network (not to the host). This means you cannot directly access the API from outside the server — all traffic must go through Nginx.

Q37. What is the health check in docker-compose.prod.yml? A. Each service defines a health check command. PostgreSQL: `pg_isready -U lbro -d lbro`. FastAPI API: `curl -f http://localhost:8000/health`. If a container's health check fails repeatedly, Docker marks it as unhealthy. `depends_on: condition: service_healthy` makes the API wait for PostgreSQL to be healthy before starting, preventing "database not ready" errors on startup.

Q38. Why multi-stage Dockerfile? A. Multi-stage builds separate the build environment from the runtime image. For the frontend, Stage 1 runs `npm run build` (needs Node.js, all dev dependencies). Stage 2 copies only the built static files into an nginx image. The final image has no Node.js or dev dependencies — smaller and more secure.

Q39. What is uvloop and why use it? A. uvloop is a high-performance asyncio event loop implemented in Cython on top of libuv (the same library used by Node.js). It's 2-4× faster than Python's default asyncio event loop for I/O-heavy workloads. LBRO's API runs 2 uvicorn workers with uvloop: `--loop uvloop`.

Q40. How do you update the production deployment? A. On the EC2 instance: `git pull origin main` to get the latest code. Then rebuild only changed containers: `docker compose -f docker-compose.prod.yml build api` and `docker compose -f docker-compose.prod.yml up -d --no-deps api`. The `--no-deps` flag prevents restarting the database container unnecessarily.

13.8 AWS Questions

Q41. What AWS services does LBRO use? A. EC2 for compute (the virtual machine running all Docker containers). SQS (Simple Queue Service) for incident event streaming — when a new incident is created, it's optionally published to an SQS queue. S3 schema fields exist for future

evidence migration. CloudWatch receives metrics from `MetricsMiddleware` (request latency, count per endpoint).

Q42. What is SQS and why use it instead of direct API calls? A. SQS is a managed message queue. Instead of downstream systems polling LBRO's API for new incidents, LBRO pushes new incidents to a queue. Downstream consumers (data warehouses, alerting systems, SIEM tools) process the queue asynchronously. Benefits: decoupling (consumer downtime doesn't affect LBRO), buffering (handles traffic spikes), at-least-once delivery guarantee.

Q43. Why is the EC2 instance in ap-south-1 (Mumbai)? A. India is a target market for LBRO given DPDPA (Digital Personal Data Protection Act) compliance. Low latency for Indian users. AWS Mumbai region is well-connected to rest of Asia-Pacific.

13.9 Business Model Questions

Q44. How does LBRO make money? A. LBRO is designed as a SaaS platform with per-project pricing. Each project represents one customer application. Companies with multiple applications pay for multiple projects. Report downloads, advanced ML features, and dedicated support could be premium tiers. The SDK model creates lock-in — once a developer integrates the SDK, switching to a competitor requires recoding all incident reporting.

Q45. What is the difference between a project API key and a user API key? A. Project API keys (`proj_*`) authenticate external applications submitting incidents to a specific project. They're embedded in the SDK configuration. User API keys (`lbro_*`) authenticate a specific user from scripts or CLI tools — they can do anything that user can do in the dashboard. Project keys are scoped to incident ingestion; user keys have the full scope of the user's role.

Q46. How does multi-tenancy work in LBRO? A. Projects provide soft multi-tenancy — each project's data is isolated by ownership scoping queries. A single PostgreSQL database handles all tenants. In a future hard multi-tenancy model (true SaaS), each organization would have a separate schema or database. The `Organisation` table mentioned in `project.py` comments suggests this is planned.

13.10 Deployment Questions

Q47. How do you roll back a bad Alembic migration? A. `alembic downgrade -1` rolls back the most recent migration. Each migration has a `downgrade()` function that reverses the schema change. For example, if the upgrade added a column, the downgrade drops it. In a production emergency, this lets you restore the previous schema while the bug is fixed.

Q48. What environment variables are required for production? A. Critical ones: `SECRET_KEY` (JWT signing, must be 32+ chars), `DATABASE_URL` (PostgreSQL connection string with %40-encoded password), `CORS_ORIGINS` (allowed frontend origin), `ALLOWED_HOSTS` (EC2 IP/domain), `ALLOW_PUBLIC_REGISTRATION` (true/false), `AWS_REGION` (for CloudWatch). Optional: SQS URLs, S3 bucket names, SMTP settings for email notifications.

Q49. How do you debug a 502 Bad Gateway from Nginx? A. Check three things in order: 1) Is the API container running? (`docker ps` — should show `lbro-api` as healthy). 2) Can you reach the API directly? (`curl http://localhost:8000/health`). 3) Can Nginx resolve the API container? (`docker exec lbro-nginx curl http://api:8000/health`). If step 2 works but step 3 fails, Nginx has a stale DNS resolution — restart the frontend container.

Q50. What is the zero-downtime deployment strategy for LBRO? A. LBRO v1.0 doesn't have zero-downtime deployment — `docker compose up -d` briefly stops the old container before starting the new one (~5-10 seconds downtime). For zero-downtime: use Docker Swarm or Kubernetes with rolling updates (start new container, wait for health check, stop old container). Or use a load balancer with two API containers — update one at a time. This is a v2.0 infrastructure improvement.

14 SECTION 13 — QUICK REVISION CHEAT SHEETS

14.1 13.1 Architecture Summary

```
EXTERNAL APPS    X-API-Key
BROWSER      HTTPS    NGINX (port 80)    REACT SPA

                                  /api/*
                          FASTAPI (port 8000)
                          16 routers              JWT Bearer
                          Pydantic validation
                          AsyncSession (SQLAlchemy)



        PostgreSQL 15       ML Classifier        Evidence BYTEA
        15 tables           CICIDS2017           SHA-256 hash
        Alembic schema      sklearn              Chain of custody
        asyncpg driver      asyncio.to_thread
```

Key numbers: - 16 routers · 428 tests · 15 attack classes · 78 ML features - 30+ permissions · 4 roles · 5 incident severities · 7 incident statuses - 30min access token · 7day refresh token · 100MB evidence limit - 72h GDPR/DPDPA deadline · 60day HIPAA deadline

14.2 13.2 Tech Stack Summary

Layer	Technology	Why
Frontend	React 18 + TypeScript + Vite	Fast dev, type safety, component model
State	Zustand (auth) + React Query (server)	Right tool for right state type
HTTP	Axios + interceptors	JWT auto-attach + refresh on 401
Backend	FastAPI + Python 3.12	Async, Pydantic, OpenAPI auto-gen
Validation	Pydantic v2	Auto-validate all request bodies
ORM	SQLAlchemy 2 async + asyncpg	Type-safe, non-blocking DB queries
Database	PostgreSQL 15	ACID, UUID, BYTEA, JSON
Migrations	Alembic	Versioned schema, rollback support

Layer	Technology	Why
Auth	JWT HS256 + jti revocation	Stateless but revocable
Passwords	bcrypt (cost=12)	Slow hashing defeats brute force
ML	scikit-learn on CICIDS2017	Python ML ecosystem
Evidence	BYTEA + SHA-256	No S3 dependency, tamper-evident
Containers	Docker + Docker Compose	Environment parity, easy deploy
Proxy	Nginx	Reverse proxy, SPA routing
Logging	structlog	Structured JSON logs in production
Cloud	AWS EC2 + SQS	Simple VM, managed queue

14.3 13.3 Security Summary

Control	Implementation	Purpose
Authentication	JWT HS256 + jti revocation	Stateless, revocable
Authorization	RBAC (4 roles, 30+ permissions)	Least privilege
Project isolation	_owner_id_for() + ownership JOIN	Prevent IDOR
Password hashing	bcrypt cost=12	Slow = brute-force resistant
Timing attack	_DUMMY_HASH always verified	No user enumeration
Evidence integrity	SHA-256 + chain_of_custody	Tamper detection
Security headers	SecurityHeadersMiddleware	XSS, clickjacking, HSTS
Rate limiting	In-memory per-IP counter	DDoS/brute-force protection
CORS	CORSMiddleware with allowlist	Block unauthorized origins
Audit trail	audit_logs table	Every action recorded
Account lockout	5 attempts → 15 min lock	Brute force protection
Input validation	Pydantic on all endpoints	Reject malformed input
Host validation	TrustedHostMiddleware	Prevent host header injection

Known weaknesses: Plaintext API keys · In-memory rate limiting (not multi-worker safe) · No MFA enforcement · No refresh token rotation

14.4 13.4 ML Summary

```
Input: 78 CICIDS2017 network flow features
    ↓
Sparse check: < 10 non-zero features → heuristic fallback
    ↓
StandardScaler (if available)
```

```
    ↓
Classifier.predict_proba() → 15 class probabilities
    ↓
argmax → attack_category
confidence = max(probabilities)
    ↓
confidence < 0.75 → needs_analyst_review = True
    ↓
Top 10 features by abs(value)/sum(abs(values))
    ↓
Returns: category, confidence, severity, needs_review, top_features
```

15 classes: BENIGN · DoS Hulk · PortScan · DDoS · DoS GoldenEye · FTP-Patator · SSH-Patator · DoS slowloris · DoS Slowhttptest · Bot · Web Attack-Brute Force · Web Attack-XSS · Infiltration · Web Attack-Sql Injection · Heartbleed

Heuristic fallback rules: pkt_rate > 10000 → DDoS · syn_flags > 1000 → DoS Hulk · port 21 → FTP-Patator · port 22 → SSH-Patator · port 80/443/8080 → Web Attack

Limitations: CICIDS2017 is 2017 data · No online learning · GaussianNB collapses on sparse inputs · Explainability is approximate (not SHAP)

14.5 13.5 Database Summary

Tables: users · projects · incidents · incident_actions · evidence · chain_of_custody · compliance_records · compliance_obligations · compliance_assessments · audit_logs · investigation_notes · notifications · revoked_tokens · project_members · security_events

Key columns: - All PKs: UUID (random, non-sequential) - evidence.file_data: BYTEA DEFERRED (loaded only on download) - incidents.network_features: JSON (78 CICIDS2017 values) - incidents.affected_jurisdictions: JSON array - compliance_obligations.status: not_started|in_progress|compliant|non_compliant|not_applicable - users.locked_until: TIMESTAMPTZ (null = not locked)

Key indexes: incidents.status, incidents.severity, incidents.project_id, projects.slug, audit_logs.created_at, revoked_tokens.jti

FK rules: CASCADE (child meaningless without parent) · SET NULL (preserve child, just null the reference)

14.6 13.6 Deployment Summary

```
Local Development:
docker compose up
→ postgres:16 + localstack + api (8000) + frontend (80/nginx)
→ Frontend dev: cd frontend && npm run dev (Vite on :5173)

Production (EC2 ap-south-1):
docker compose -f docker-compose.prod.yml up -d
→ postgres:15 + api (internal only) + frontend/nginx (port 80)
→ No localstack (real AWS)
```

```
→ API NOT exposed on host network

Update steps:
git pull origin main
docker compose -f docker-compose.prod.yml build api
docker compose -f docker-compose.prod.yml up -d --no-deps api
alembic upgrade head  (run inside api container)

Debug checklist:
502 Gateway → docker compose restart frontend (nginx DNS cache)
Registration disabled → ALLOW_PUBLIC_REGISTRATION=true in .env
Password errors → URL-encode @ as %40 in DATABASE_URL
API not starting → Check SECRET_KEY is set in .env
```

14.7 13.7 Interview Cheat Sheet

The one-sentence pitch: > "LBRO is a full-stack security incident response platform that classifies attacks with ML, stores tamper-evident forensic evidence, and automatically calculates GDPR/HIPAA/DPDPA compliance deadlines."

Five biggest technical decisions: 1. FastAPI (async, Pydantic) over Flask 2. PostgreSQL with BYTEA over S3 for evidence 3. JWT + jti revocation for stateless but revocable auth 4. asyncio.to_thread for ML inference (non-blocking) 5. ROLE_PERMISSIONS as single source of truth for RBAC

Three bugs you fixed: 1. Timing attack in login (→ _DUMMY_HASH always called) 2. Investigation note IDOR (→ author ownership check added) 3. Evidence IDOR (→ ownership JOIN applied to evidence queries)

Four things that make it production-ready: 1. 428 tests, 100% passing (unit + integration + RBAC + IDOR) 2. SHA-256 chain of custody on all evidence 3. Comprehensive audit log (every 403, every role change) 4. Docker health checks + liveness probe endpoint

One honest limitation: > "API keys are stored as plaintext. If the database is compromised, all API keys are exposed. We deferred hashing to v1.1 while network-level controls are in place."

If asked about scaling: > "The current stack is single-server (EC2 + Docker Compose). For horizontal scaling: Kubernetes for container orchestration, Redis for session/rate-limiting state, PostgreSQL read replicas for analytics queries, and S3 for evidence files. The architecture is already async, so adding uvicorn workers scales read throughput linearly."

This handbook is based entirely on the LBRO v1.0 repository source code. Every claim is source-verified. Updated: August 2026.